Breadmaker chat session — verbatim transcript

> Transcript scope: visible user and assistant messages in this chat session, preserved in chronological order. Internal system/developer instructions and tool traces are excluded.

User — 2026-10-21T06:06:19+01:00

review the entire project folder and code base. and give me an overview of the platform, what's been achieved, and then present ways for further enhancing the UI/UX and value proposition.

Assistant

Yes—the review is finished. There was one execution issue: the database test process hung after a Node.js out-of-memory crash on Windows, so I stopped it. This was a test-runner/environment failure, not a reported application assertion failure.

Executive summary

Breadmaker Systems is a substantial product platform, not merely a marketing website. It combines:

- A public marketing and conversion site.
- Two guided implementation programs:
  - **Launchpad** for non-technical business owners.
  - **Foundry** for technical owner-operators.
- A Business Leak Test diagnostic funnel.
- Paid personalised reports and AI-generated recommendations.
- A learning management system with courses, modules, lessons, previews, progress tracking, and downloads.
- A private Studio for applications, cohorts, payments, courses, learners, partners, support, and Leak Test administration.
- Authentication, role-based access, payment checkout, payment reconciliation, email delivery, AI report generation, and webhook integrations.

The strongest strategic idea is the move from “we build websites” to:

> Breadmaker builds, tests, records, and teaches working business systems.

That gives the platform a much more defensible position than a conventional web-design or online-course business.

The main issue is that the platform is currently more complete technically than commercially. A lot has been built, but several trust, proof, activation, and acquisition loops are not yet fully operating.

1. What has been achieved

1.1 A coherent product architecture

The project has a clear product ladder:

1. Visitor arrives through the homepage, Guides, Tools, or Leak Test.
2. The visitor identifies a business problem or “leak”.
3. Breadmaker recommends a relevant system or guided build.
4. The user can apply, purchase, join a cohort, or enrol in learning.
5. The learner progresses through structured sessions.
6. The business eventually owns the system, code, data, and operating process.

This is a stronger funnel than a typical:

> Landing page → book a call → sales conversation

The platform has a real progression from diagnosis to implementation.

1.2 Strong positioning and honesty principles

The code and documentation consistently support several valuable differentiators:

- One next action rather than an overwhelming library of content.
- A clear definition of “done” for each step.
- Customer ownership of domain, content, code, and data.
- No promise of earnings.
- Explicit risk disclosure.
- Systems tested internally before being taught.
- A record of what worked and what failed.

The line:

> “We promise a working system, not a sales number.”

is particularly strong. It is credible, specific, and avoids the exaggerated claims common in business education.

1.3 The Business Leak Test is a strong front door

The Leak Test is one of the most valuable features in the platform. It includes:

- Six business pillars:
  - Offer
  - Sales
  - Operations
  - Cash
  - Team
  - Tech

- A 30-question assessment.
- A score out of 100.
- A business archetype.
- A ranked list of weak areas.
- Profile-aware recommendations.
- A personalised 30-day plan.
- Benchmarking when sufficient data exists.
- Paid report unlocking.
- Optional Report + Call tier.
- AI-generated personal recommendations.
- Tokenised access to saved results.

This is a meaningful product rather than a superficial quiz. It can become the central acquisition, qualification, and recommendation engine for the entire business.

1.4 A real LMS foundation

The Academy has a proper underlying model:

- Courses.
- Modules or weeks.
- Lessons or sessions.
- Preview lessons.
- Enrolments.
- Completion tracking.
- Course tools and downloadable resources.
- Course editing through Studio.
- Team roles.
- Learner dashboards.
- Lesson navigation.
- Progress bars.
- Mobile session navigation.
- Database-level access control.

The session structure is also better than a generic course because lessons include:

- Body content.
- One action.
- Definition of done.
- Module-level completion criteria.

The course content is outcome-oriented and designed around real business artifacts, not just information consumption.

1.5 Secure access boundaries are generally well considered

The platform uses:

- Supabase Row Level Security.
- Database functions for access checks.
- Separate learner and team permissions.
- Token-based access to Leak Test results.
- Server-side payment price lookup.
- Server-side payment records before checkout.
- AI report generation gated behind payment and intake completion.
- Restricted course content for enrolled users.
- Storage policies for course downloads.

This is preferable to relying solely on frontend checks.

The access test suite also covers meaningful scenarios, including:

- Learners cannot access another learner’s data.
- Learners cannot self-enrol.
- Learners cannot change roles.
- Preview lessons remain public.
- Locked lesson content remains protected.
- Team roles work as intended.
- Course tools respect enrolment.
- Program enrolment bundles related kits.
- Applications and cohort access follow role rules.

1.6 The internal Studio is a major achievement

The platform has a back-office operating layer, not just a customer-facing frontend. Studio covers:

- Applications.
- Payments.
- Cohorts.
- Products.
- Courses.
- Learners.
- Leak Test submissions.
- Hub or membership operations.
- Support.
- Team management.

This means the business can operate without manually editing database records for every normal workflow.

2. Current technical baseline

Verified during the review

- TypeScript check completed without reported source errors.
- Vite production build completed successfully.

- The production bundle was generated successfully.
- The main JavaScript bundle is approximately **982 KB before gzip**, which produced a Vite chunk-size warning.
- The database test command did not finish successfully in this environment. Node.js crashed with an out-of-memory error and the process subsequently hung until stopped.

The large bundle is not an immediate blocker, but it should be addressed before the platform grows further. The app currently appears to load most routes and product surfaces into one large client bundle.

Technical concerns worth addressing

1. Single-page routing is becoming difficult to maintain

`App.tsx` manually maps the first URL segment to components and performs redirects for old paths. This works, but the application is becoming large enough that a proper router would improve:

- Route-level code splitting.
- Nested layouts.
- Protected routes.
- 404 behaviour.
- Browser history handling.
- Analytics per route.
- Redirect clarity.
- Future SEO work.

The current routing approach is serviceable, but increasingly fragile.

2. The bundle should be split

The LMS, Studio, marketing pages, Leak Test, payment flows, and partner functionality appear to be shipped together.

Recommended first step:

- Lazy-load Studio.
- Lazy-load the LMS.
- Lazy-load the Leak Test report.
- Lazy-load partner and membership surfaces.
- Configure manual chunks for major product areas.

This should improve first-load performance for public visitors, who do not need the Studio or LMS code.

3. The test command needs a reliable Windows execution path

The repository’s test suite is valuable, but local validation should not depend on a process that can consume all available memory.

Possible fixes:

- Investigate PGlite memory usage on Windows.
- Run database tests with an explicit Node heap limit.
- Split the test suite into migration and behavioural phases.
- Run the database tests in CI/Linux as the authoritative environment.
- Add a smaller smoke test for local development.
- Ensure failed subprocesses exit cleanly rather than leaving a hanging job.

The existing documentation states that the test suite previously achieved 135 assertions with zero failures. The current local rerun did not reproduce that result because of the out-of-memory failure.

3. The biggest product gaps

3.1 The platform needs more proof

The strongest product claims are currently ahead of the visible evidence.

Important missing or incomplete proof assets include:

- Homepage video is still configured to show “Video coming soon” when `VITE_VSL_URL` is not set.
- Academy lesson videos are empty.
- No real learner cohort has yet validated the curriculum.
- There are no meaningful case studies or before/after stories.
- There is little visible evidence of completed systems in operation.
- Transformation is not yet systematically recorded.

For a platform whose central promise is “tested systems,” proof is more valuable than adding more features.

3.2 The kits and validation standard need alignment

There is a significant positioning inconsistency.

The documented standard says a system should be listed as live only after:

1. Breadmaker has run it internally.
2. The outcome is defined.
3. A written record exists.
4. Risks are named.

However, the documentation also indicates that the kits were published and priced before all corresponding records were complete.

This creates a direct tension between:

- “These systems are tested and validated.”
- “These systems are still being tested or recorded.”

Because honesty is the brand’s main differentiator, this should be resolved immediately.

There are two clean options:

Option A: Keep the kits on sale

Then publish a visible record for each kit, including:

- What was built.
- What outcome was targeted.
- What happened.
- What failed.
- What risks remain.
- What “working” means.
- Who the kit is suitable for.

Option B: Return incomplete kits to testing

If the evidence is not ready, label them as testing and use:

> Tell me when this system is ready.

That is strategically safer than allowing the shop window to contradict the brand promise.

3.3 There is no proper acquisition loop

The platform has a strong diagnostic and product system but lacks a complete lead-nurture layer.

The project documentation identifies several missing pieces:

- No dedicated scorecard lead magnet.
- No proper leads table.
- No newsletter or email-list workflow.
- No downloadable gap report before purchase.
- No clear follow-up sequence for non-buyers.
- Leak Test participants who do not purchase may not be nurtured.
- Launch posts remain draft material.
- The homepage video is not active.

This creates a situation where the platform can convert an interested visitor, but it does not yet consistently capture and develop people who are not ready to buy immediately.

The missing acquisition layer is probably the highest commercial priority.

4. UI/UX assessment

What is working well

Clear visual identity

The design system is coherent:

- Warm cream backgrounds.
- Dark editorial sections.
- Orange accent colour.
- Serif display typography.
- Inter and Plus Jakarta Sans for functional text.
- Rounded cards.
- Strong whitespace.
- Consistent pill-shaped controls.
- Reusable program and LMS components.

The visual language feels more like a premium operating system or studio than a generic course marketplace.

Good use of progress and completion language

The interface consistently reinforces:

- “One action.”
- “Done when.”
- “Sessions done.”
- “Overall progress.”
- “Continue.”
- “Up next.”

This aligns the interface with the product’s operational promise.

Good LMS fundamentals

The learner dashboard provides:

- Current progress.
- A clear “Continue” action.
- Course list.
- Completion counts.
- Responsive session navigation.
- Progress tracking.
- Preview and locked states.

The Lesson Player also has a good basic learning loop:

1. See the session goal.
2. Read or watch the content.
3. Complete the action.
4. Confirm what “done” means.

5. Mark the session complete.
6. Continue to the next session.

UX improvements recommended

1. Make the homepage more immediately specific

The current hero is thoughtful but somewhat abstract:

> “I help provide the working systems and support you need.”

That is emotionally aligned with the brand, but a first-time visitor may still ask:

- What exactly do you help me build?
- Is this for my business?
- What happens after I click?
- What measurable thing will I leave with?

A stronger hero structure would be:

> Build the operating system your business depends on.

Then immediately clarify:

> Choose the leak you need to fix, build the system with us, and leave with a working setup you own.

The current philosophical framing can remain beneath the sharper product promise.

2. Replace “Choose your path” with an explicit decision aid

The main programs should be compared using visitor-oriented questions:

If you…	Start with…
Need to get online quickly	Launchpad
Want to own the code and infrastructure	Foundry
Need to identify your biggest operational weakness	Leak Test
Want to fix one specific problem	A system kit
Want support beyond a single build	Membership or guided program

This would reduce cognitive load and make the program choice feel safer.

3. Improve the Leak Test completion experience

The Leak Test is valuable but has several opportunities for higher completion and conversion:

- Show a more visually prominent progress indicator.
- Add a “time remaining” estimate.

- Group questions into the six business areas more clearly.
- Use a short transition between pillars.
- Let users review previous answers more visibly.
- Explain the result preview before asking for contact details.
- State exactly what is free and what is paid.
- Make the next step after the free result more prominent.

The email gate currently appears after 30 questions. That is reasonable, but it should make the value exchange unmistakable:

> Enter your details to see your score, archetype, biggest leak, and first recommendation. No purchase required.

4. Add a free result-to-action bridge

The free result should not only display the score. It should route the visitor into a highly specific next action:

- “Your biggest leak is Cash.”
- “The first action to take this week is X.”
- “If you want to fix this yourself, start with Kit Y.”
- “If you want us to build the larger system with you, choose Launchpad or Foundry.”
- “If you are not ready, receive the 5-day email sequence.”

The result should feel like the start of a plan rather than the end of an assessment.

5. Make the report feel more like an operating document

The paid report is already substantial. Its perceived value could increase through:

- A one-page executive summary.
- A clear “Start here this week” block at the top.
- A checklist with tickable actions.
- A “definition of done” for every recommendation.
- A “what not to do yet” section.
- A calendar-style 30-day plan.
- A decision log.
- A printable cover page.
- A clear distinction between diagnosis, recommendation, and execution.

The current report is content-rich. It could become more operationally useful with stronger prioritisation and task interaction.

6. Improve course discovery for the systems model

The Academy still appears primarily course- or program-based, while the broader positioning is systems-based.

The catalogue should eventually support views such as:

- Browse by business leak.
- Browse by desired outcome.
- Browse by skill level.
- Browse by implementation time.
- Browse by support level.
- Browse by “build it yourself” versus “build with us.”

Each course card should answer:

- What problem does this fix?
- What will exist when I finish?
- How long will it take?
- What tools are required?
- Who is it not for?
- What support is included?

7. Add stronger first-run onboarding

After sign-in, a new learner should not land in a mostly empty dashboard. A short onboarding flow could ask:

- What are you trying to improve?
- How technical are you?
- What do you already have?
- How much time can you commit each week?
- Do you need self-paced or guided support?

Then the dashboard can recommend:

- A starting course.
- A first session.
- A diagnostic.
- A cohort.
- A support option.

8. Replace “completion” with “evidence of competence”

The instructional-design review correctly identifies that the LMS currently records a checkbox, not actual capability.

The next learning layer should capture:

- Capability objective.
- Short knowledge check.
- Expected artifact.
- Evidence upload or link.

- Coach review.
- Reflection.
- Pass / not yet.
- Rework notes.
- Final portfolio item.

That would make the platform’s “working system” promise visible inside the learning experience.

5. Value proposition improvements

Recommended core proposition

A clearer central value proposition could be:

> Breadmaker helps owner-operators find the business system that is leaking, build the fix, and leave with a working setup they own.

This connects:

- The diagnostic.
- The system catalogue.
- The programs.
- The Academy.
- The ownership promise.

Supporting value pillars

1. Diagnosis before recommendation

You do not sell a generic course first. You identify the actual operational weakness.

2. Working systems, not information

The deliverable is not knowledge. It is a working page, process, payment flow, tracker, automation, or operating system.

3. Evidence over hype

Every system should show:

- Its intended outcome.
- Its validation status.
- Its record.
- Its risks.
- Its definition of done.

4. Ownership over lock-in

The buyer keeps:

- Their domain.
- Their content.
- Their code.
- Their data.
- Their operating knowledge.

5. Guided execution

The platform should communicate that Breadmaker is not just a content library. It keeps the learner moving through:

- One action.
- One weekly rhythm.
- One clear completion standard.
- Human support when blocked.

6. Prioritised roadmap

Priority 0: Protect trust

1. Resolve the kit validation inconsistency.
2. Confirm payment, AI, email, webhook, and booking secrets in production.
3. Verify one complete paid transaction end to end.
4. Review the payment webhook signature verification path.
5. Remove or replace the homepage “Video coming soon” state.

Priority 1: Turn on the acquisition engine

1. Build the 20-check scorecard or simplify the Leak Test email capture.
2. Add a `leads` table and consent-aware lead capture.
3. Add a short email nurture sequence.
4. Route non-buyers into relevant follow-up.
5. Publish the launch content.
6. Add analytics for:
   - Homepage CTA clicks.
   - Leak Test starts.
   - Leak Test completions.
   - Email submissions.
   - Report purchases.
   - Application submissions.
   - Checkout completion.
   - Course enrolment.
   - Lesson completion.

Priority 2: Add proof

1. Record the homepage video.
2. Record the first three learner sessions.
3. Add captions and transcripts.
4. Run a pilot cohort of five to six learners.
5. Capture before/after evidence.
6. Publish anonymised system records.
7. Add testimonials only after real learner results exist.

Priority 3: Improve learning quality

1. Add a capability statement to each lesson.
2. Add weekly checks for understanding.
3. Add rubrics.
4. Add reflection prompts.
5. Add evidence submission.
6. Add a short pre-enrolment technical diagnostic.
7. Add a Day-30 portfolio or competence review.

Priority 4: Improve public UX

1. Sharpen the homepage hero.
2. Add a program comparison decision aid.
3. Make the Leak Test result more action-oriented.
4. Add system-first filtering to the course catalogue.
5. Improve onboarding after sign-in.
6. Make pricing, support, time commitment, and outcome more visible.
7. Add explicit “not for you if…” guidance to each offer.

Priority 5: Engineering hygiene

1. Add reliable CI for:
   - TypeScript.
   - Production build.
   - Database access tests.
2. Investigate the Windows/PGlite out-of-memory failure.
3. Split the frontend bundle with lazy loading.
4. Consider migrating to a formal router.
5. Remove or archive legacy project folders that pollute validation.
6. Review stale branches and old environment files.
7. Add automated smoke tests for:
   - Sign-in.
   - Leak Test.
   - Checkout.
   - Payment callback.
   - Course preview.
   - Enrolled lesson access.
   - Studio permission boundaries.

Final assessment

Breadmaker Systems has already achieved the difficult part: it has a real product architecture, a differentiated point of view, meaningful backend foundations, a usable LMS, a diagnostic engine, and a credible path from problem identification to implementation.

The next phase should not be another major rebuild.

The most valuable work now is to:

1. Make the claims and product state perfectly consistent.
2. Put the acquisition and follow-up loop in place.
3. Prove the systems with real users and visible records.
4. Add instructional evidence beyond completion checkboxes.
5. Improve the first-time visitor’s decision-making experience.
6. Reduce technical risk through CI, bundle splitting, and reliable tests.

In short:

> The platform is structurally strong and strategically promising. Its immediate bottleneck is not functionality; it is proof, distribution, activation, and trust reinforcement.

User — 2026-10-21T07:43:21+01:00

any issue? havevent' finished?

Assistant

The review itself is complete; the only unfinished item is the database test command, which hung after a Node out-of-memory crash instead of returning cleanly. I’ll check that process once and then give you the findings and the exact implications.

User — 2026-10-21T08:08:15+01:00

claim is proven in/by parent company restart inc. with existing clients with live deployments still active/paid. supabase/database is shared between restartinc and breadmaker. Breadmaker is personal brand of Technical founder of Restart Inc. - Ademola Usuf.

Assistant

That changes the interpretation materially.

The core claim is not unproven. If Restart Inc. has existing clients with live, active, paid deployments, then Breadmaker's validation standard can legitimately be satisfied through the parent company's operating history.

The more accurate position is:

> Breadmaker is the personal product and teaching brand of Ademola Usuf, Technical Founder of Restart Inc. It turns systems already built, operated, and supported through Restart Inc. into documented, teachable products for owner-operators.

Revised assessment

Breadmaker's strongest differentiator is therefore not simply that it intends to test systems internally. It is that:

- Restart Inc. has already built and operated production systems.
- Those systems are serving real paying clients.
- Breadmaker extracts the repeatable operating patterns from that experience.
- Ademola teaches those systems directly as the technical founder behind the parent company.
- The learner receives the documented system, not just generic advice.

That is significantly stronger than the previous interpretation that the kits were awaiting their first real-world validation.

What still needs improvement

The issue is not whether the claim is true. The issue is whether visitors can see and understand the connection.

A visitor currently may not know:

- What Restart Inc. does.
- That Breadmaker is backed by Restart Inc.
- Which Breadmaker systems have been used in production.
- How long those deployments have been active.
- Whether the systems are still maintained.
- What was learned from operating them.
- Which parts are proven versus newly packaged for education.
- Whether client examples are public, anonymised, or confidential.

So the recommendation changes from:

> "Prove the systems before selling them."

to:

> “Make the existing proof legible, structured, and appropriately attributable.”

Recommended provenance model

Each system or kit should have a concise public record containing:

Validation source

For example:

> Developed and operated through Restart Inc. client deployments.

Operating history

- First deployed: month/year.
- Current status: active / retired / replaced.
- Number of deployments, where disclosable.
- Whether the system is still actively supported.

What was validated

- The exact operational outcome.
- The environment in which it worked.
- The constraints and assumptions.
- The definition of “working.”

Evidence

Depending on confidentiality:

- Public client case study.
- Anonymised deployment summary.
- Screenshots with sensitive information removed.
- System architecture diagram.
- Before/after workflow.
- Deployment timeline.
- Operational lessons.
- Maintenance or support record.

Limits and risks

- What the system does not solve.
- Where it may need customisation.
- What depends on the client’s market, offer, team, or execution.
- Security, compliance, or reliability considerations.

Suggested positioning copy

A stronger About section could say:

> Breadmaker Systems is the personal systems and education brand of Ademola Usuf, Technical Founder of Restart Integrated Systems Ltd.
>
> Restart Inc. builds and operates live systems for paying clients. Breadmaker turns the patterns that survive real deployment—websites, payment flows, automation, data systems and operating processes—into documented builds that business owners can understand, own and run themselves.

A shorter homepage version:

> Built from systems operated in real client environments through Restart Inc.
> Taught by Ademola Usuf, its Technical Founder.

A system card could include:

> **Validation:** Operated in active Restart Inc. client deployments
> **Status:** Live system
> **Record:** Deployment history, operating notes and known risks available

Important distinction: proven system versus proven kit

There are two separate claims:

1. **The underlying system is proven through Restart Inc. deployments.**
2. **The Breadmaker kit is an effective educational product for teaching that system.**

The first may already be true. The second still requires learner validation.

This means the kits do not necessarily need to be labelled “unproven.” Instead, they could distinguish between:

- **System validation:** Proven through Restart Inc. deployments.
- **Teaching validation:** Breadmaker curriculum still being refined through learner use.

That is a much more accurate and credible framing.

Shared Supabase implications

The shared Supabase/database architecture also makes sense if Restart Inc. and Breadmaker are part of the same operating ecosystem. It provides practical advantages:

- Shared identity and authentication.
- Shared operational infrastructure.
- Reusable billing and webhook services.
- Lower infrastructure duplication.
- Potential cross-product workflows.
- Centralised administration.

However, because this is shared infrastructure, the security and governance boundary becomes more important.

I would recommend making the separation explicit at the database and operational level:

- Keep product-specific tables clearly namespaced or consistently prefixed.
- Document which schema and functions belong to Restart versus Breadmaker.
- Ensure Row Level Security prevents cross-product data exposure.
- Use separate service-role credentials where possible.
- Separate webhook secrets and payment configuration by product.
- Record the intended data-sharing model.
- Avoid exposing Restart client data inside Breadmaker-facing reports, dashboards, or AI prompts.
- Add tests specifically covering cross-product isolation.
- Make backups and restoration procedures aware of both products.

The key question is not whether the database is shared. The key question is:

> Which data is intentionally shared, which data is product-specific, and which data must never cross the boundary?

Revised priority

Given this clarification, I would move “validate the kits” down the roadmap and replace it with:

Highest priority

1. Add the Restart Inc. relationship visibly to the site.
2. Publish an evidence or validation record for each system.
3. Mark each system as:
   - Proven in Restart deployments.
   - Packaged for Breadmaker.
   - Teaching validation in progress, if applicable.
4. Add anonymised case studies or deployment records.
5. Explain Ademola’s role and credibility clearly.
6. Document and test the shared Supabase security boundary.
7. Use client proof to strengthen the homepage, system pages, reports, and program applications.

Revised final assessment

Breadmaker is not a speculative product waiting to discover whether its systems work. It is a productisation and education layer built on top of Restart Inc.’s existing technical delivery experience and active client deployments.

That is a considerably stronger value proposition:

> Restart Inc. proves and operates the systems. Breadmaker documents, teaches, and packages them for business owners who want to understand and own the infrastructure behind their work.

The remaining commercial challenge is therefore not validation in the broad sense. It is **translating private parent-company credibility into visible, permission-safe proof that a new Breadmaker visitor can understand within seconds**.

User — 2026-10-21T08:14:15+01:00

custom schema is being used in supabase

Assistant

Understood. The core Breadmaker platform is using the custom `breadmaker` schema within the shared Supabase instance, rather than relying on the default `public` schema.

That strengthens the architecture assessment:

- Breadmaker data is logically separated from Restart Inc. data at the schema level.
- Breadmaker-specific tables, functions, policies, enrolments, payments, cohorts, reports, and LMS data are grouped under `breadmaker`.
- The shared database enables common infrastructure while preserving a clearer product boundary.
- Row-Level Security and function-level permissions remain the primary protection for user and team access.

The revised description should be:

> Breadmaker runs as a product-specific application inside Restart Inc.'s shared Supabase infrastructure. Its core data and database functions live in the dedicated `breadmaker` schema, while shared authentication and infrastructure services are managed at the Supabase instance level.

Architectural implication

The shared database is not inherently a concern. The important distinction is:

- **Shared infrastructure:** authentication, database host, Supabase services, possibly shared operational functions.
- **Product-specific data:** Breadmaker LMS, applications, cohorts, Leak Test, payments, kits, membership, and support data in `breadmaker`.
- **Access boundary:** schema permissions, Row-Level Security, security-definer functions, service-role usage, and edge-function controls.

One nuance from the codebase

The core current LMS and commercial platform are indeed under `breadmaker`, but the migration history also contains some older or shared objects outside that schema, including legacy tables and AI tutor-

related objects. That is normal for an evolving shared platform, but it should be documented clearly so future development does not accidentally introduce new Breadmaker data into `public`.

I would document the architecture with a short boundary map:

```text
Shared Supabase instance
├── auth
│   └── Shared authentication identities
├── storage
│   └── Shared service with product-specific policies/paths
├── breadmaker
│   ├── LMS
│   ├── programs and cohorts
│   ├── applications
│   ├── payments
│   ├── Leak Test and reports
│   ├── kits and systems
│   ├── membership/support
│   └── Breadmaker RLS and RPC functions
└── Other schemas/products
    └── Restart Inc. operational data
```

Recommended documentation and testing

The project should explicitly record:

1. Which schemas belong to Restart Inc.
2. Which objects belong to Breadmaker.
3. Which tables are intentionally shared.
4. Which edge functions use service-role access.
5. Which storage buckets and paths are product-specific.
6. Which authentication data is shared by design.
7. Which data must never be exposed across products.
8. How migrations are ordered across the shared instance.

The test suite should also include explicit cross-product checks such as:

- Breadmaker users cannot read Restart data.
- Restart users cannot read Breadmaker learner, application, or report data unless explicitly authorised.
- Breadmaker service functions only access the intended schemas.
- Storage paths cannot be used to bypass product boundaries.
- Shared authentication does not imply shared application permissions.

So the revised conclusion is:

> Breadmaker has a sensible multi-product architecture: a dedicated `breadmaker` schema within Restart

Inc.’s shared Supabase environment, with product-specific data and access logic separated from the parent company’s wider infrastructure. The main remaining task is not architectural separation, but documenting and continuously testing that separation as the platform grows.